DESIGN NOTE OPERATIONS

One worker, many threads, and what Redis is actually for

/metrics is per process, so gunicorn runs one worker and scales with threads; capacity comes from more instances. Redis then gives you one response cache, one rate-limit budget and one event stream, and if it blinks, limits are never lifted.

11 October 2026 · 1 min read

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=one-worker-many-threads. This copy was made on 4 November 2026.

Two deployment questions come up more than any others: why does the recommended gunicorn command run a single worker, and do I need Redis?

One worker

`/metrics` is per process. Prometheus scrapes one process and sees that process's counters. With four gunicorn workers behind one port, each scrape lands on a random one and the numbers jump around.

So the recommended shape is one worker with several threads:

```shell
gunicorn --workers 1 --worker-class gthread --threads 8 --timeout 120 "queryapigate.app:create_app()"
```

Queries spend most of their time waiting on the database, so threads do well here. When one process is not enough, run more instances, each with its own port and its own `/metrics`, behind a reverse proxy. Prometheus scrapes them all and the bundled Grafana dashboard sums across instances.

What Redis does

Nothing, until you have more than one instance. Then it does three things:

- **One response cache.** A saved query with `cache_ttl` is warmed by whichever instance answered first and served from cache by all of them. Without Redis each process has its own cache.
- **One rate-limit budget.** A key limited to 60 requests a minute gets 60 across the fleet, counted as an atomic token bucket on Redis's clock, not 60 per instance.
- **One event stream.** `queryapigate events` reads every instance's runs.

If Redis blinks

A cached response is a miss. Limits are counted per process for a few seconds, then Redis is tried again. Limits are never lifted, and never turned into refusals. The fallback is counted in `queryapigate_rate_limit_fallbacks_total` and raises the `rate_limits_not_shared` alert so you know it happened.

The reference deployments

`deploy/scale-out/` runs three instances, PostgreSQL, Redis, the events server and Caddy with Compose. `deploy/kubernetes/` is the same with probes, a PodDisruptionBudget and zero-downtime rolling updates. Both run end to end in CI, including failover and a rolling update under traffic. Guide 42 walks through both.